

LAPORAN PRAKTIKUM 1
ALGORITMA DAN PEMROGRAMAN KOMPUTER 2
Kompleksitas Algoritma



Disusun Oleh :
Ahmad Satriyo (5002251067)

Asisten Laboratorium :
1. Muhammad Fikri Hidayat (5002241119)
2. Laudiya Meysinta Dewi (5002241123)

Dosen Pengampu :
Dr. Dwi Ratna Sulistyaningrum
19690405 199403 2 003

DEPARTEMEN MATEMATIKA
FAKULTAS SAINS DAN ANALITIKA DATA
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA
2026

BAB I

DESKRIPSI

1.1 Deskripsi Tugas

1.1.1 Tugas 1

Pada tugas pertama, mahasiswa diberikan sebuah penggalan program berupa sebuah *method* dengan label/nama ‘hitungKombinasi” yang akan menerima sebuah masukan berupa bilangan *integer* dengan keluaran atau *return* berupa bilangan *integer*. Dari penggalan program tersebut, mahasiswa diberi tugas untuk menentukan kompleksitas waktu dan Big-O dari program tersebut. Program tersebut adalah sebagai berikut:

```
public static int hitungKombinasi(int n) {  
    int kombinasi = 0;  
    for (int w = 0; w <= n; w++) {  
        for (int x = 0; x <= n; x++) {  
            for (int y = 0; y <= n; y++) {  
                for (int z = 0; z <= n; z++) {  
                    if (w + x + y + z == n) {  
                        kombinasi++;  
                    }  
                }  
            }  
        }  
    }  
    return kombinasi;  
}
```

Untuk menyelesaikan permasalahan tersebut, perlu diketahui terlebih dahulu pengertian kompleksitas waktu dan Big-O pada suatu program atau algoritma. Kompleksitas waktu $T(n)$ merupakan jumlah tahapan komputasi yang diperlukan oleh sebuah program untuk menjalankan sebuah algoritma dengan masukan sejumlah n . Sedangkan Big-O merupakan sebuah versi dari kompleksitas waktu sebuah algoritma dengan mempertimbangkan skenario terburuk dari waktu berjalannya suatu algoritma yang bergantung pada suatu konstanta atau secara formal didefinisikan sebagai:

Suatu fungsi $f(n)$ dapat dikatakan $O(g(n))$, jika dan hanya jika terdapat suatu konstanta $C > 0$ dan n_0 sedemikian sehingga:

$$f(n) \leq C \cdot g(n), \forall n \geq n_0$$

Dalam suatu program/algoritma, Big-O dari program tersebut dapat diketahui dengan meninjau berapa banyak sebuah *assignment* di dalam sebuah perulangan dikerjakan apabila diberi masukan berupa sebuah bilangan n . Pada program tersebut, terdapat fungsi perulangan bertingkat/*nested loop* berjumlah 4 dengan masing-masing variabel yang berperan sebagai variabel yang akan di-*increment* pada perulangan dimulai dari 0 hingga nilainya sama dengan n . Maka, pada masing-masing perulangan, apabila diberi sebuah masukan n , perulangan tersebut akan berjalan sebanyak $n + 1$ kali (dikarenakan 0 dan n masih termasuk dalam perulangan).

Dikarenakan terdapat 4 buah perulangan dengan masing-masing variabel *increment* bermulai dari 0 hingga n , maka untuk mengetahui kompleksitas waktu yang diperlukan program tersebut dapat ditentukan dengan mengalikan banyak perulangan yang dilakukan pada masing-masing perulangan apabila diberi masukan n atau sebagai berikut:

$$T(n) = N_1 N_2 N_3 N_4 c$$

$$T(n) = (n + 1)(n + 1)(n + 1)(n + 1)c$$

$$T(n) = cn^4 + 4cn^3 + 6cn^2 + 4cn + 1c$$

$$T(n) \approx O(n^4)$$

N pada persamaan tersebut merupakan jumlah perulangan yang dilakukan tiap perulangan saat diberikan masukan n dan c merupakan sebuah *assignment* yang dikerjakan program setiap kali perulangan berhasil dijalankan, yakni penambahan nilai variabel kombinasi setiap kondisi pada *if* terpenuhi.

Pada perhitungan Big-O, dikarenakan dimisalkan pada persamaan kompleksitas waktu sebuah algoritma dengan diberikan masukan n mendekati tak hingga, maka hanya diperhatikan derajat atau pangkat tertinggi pada persamaan tersebut. Kemudian, dikarenakan Big-O merupakan bentuk persamaan kompleksitas waktu dalam bentuk paling sederhana, maka konstanta seperti c dan angka lainnya dapat dihiraukan.

Apabila algoritma tersebut dijalankan dengan diberikan sebuah masukan berupa n , maka akan diperoleh banyak *assignment* c dijalankan atau banyak perulangan berhasil dijalankan sebagai berikut:

n	c
0	1
1	16
2	81
3	256
4	625

Apabila pola tersebut dianalisis, maka akan membentuk sebuah pola jumlah iterasi berupa:

$$\text{jumlah iterasi} = (n + 1)^4$$

Yang di mana, pola tersebut merupakan pola yang sama dalam kompleksitas waktu yang telah ditemukan sebelumnya.

1.1.2 Tugas 2

Pada tugas kedua, mahasiswa diberikan sebuah penggalan program yang sama berupa sebuah *method* dengan label/nama ‘hitungKombinasi’ yang akan menerima sebuah masukan berupa bilangan *integer* dengan keluaran atau *return* berupa bilangan *integer*. Dari penggalan program tersebut, mahasiswa diberi tugas untuk melakukan

modifikasi pada program tersebut sedemikian rupa hingga memiliki kompleksitas waktu $T(n)$ mendekati $O(1)$. Program tersebut adalah sebagai berikut:

```
public static int hitungKombinasi(int n) {
    int kombinasi = 0;
    for (int w = 0; w <= n; w++) {
        for (int x = 0; x <= n; x++) {
            for (int y = 0; y <= n; y++) {
                for (int z = 0; z <= n; z++) {
                    if (w + x + y + z == n) {
                        kombinasi++;
                    }
                }
            }
        }
    }
    return kombinasi;
}
```

Program tersebut merupakan sebuah program yang akan menghitung banyak kemungkinan penjumlahan 4 angka untuk menjadi sama dengan bilangan yang dimasukkan/*input*, yakni n dengan syarat jika w , x , y , dan z merupakan bilangan bulat yang bernilai lebih dari atau sama dengan 0 dan bernilai kurang dari atau sama dengan bilangan yang dimasukkan (n).

Salah satu pendekatan untuk memecahkan masalah tersebut adalah menggunakan teorema bintang dan garis atau *stars and bars*. Teorema bintang dan garis adalah sebuah metode kombinatorik untuk menghitung kemungkinan/cara untuk mendistribusikan n objek ke beberapa kotak yang diwakilkan dengan k .

Metode ini dilakukan dengan mewakili n jumlah benda yang ingin didistribusikan menjadi n buah bintang yang sama. Kemudian, k sebagai kotak penampung bintang akan dipisah menggunakan garis atau *bars* berjumlah $k - 1$. Kemudian, untuk menghitung kemungkinan yang ada, dapat digunakan metode pergeseran garis atau pemindahan bintang sehingga jumlah bintang tetap memenuhi syarat, yakni berjumlah n . Setiap

bintang dalam kotak-kotak tersebut kemudian dihitung dan dijumlahkan. Apabila jumlahnya sama dengan n , maka pola tersebut memenuhi syarat dan merupakan salah satu cara/kombinasi yang mungkin.

Contohnya, apabila kita memiliki angka berupa 4 yang ingin kita bagi ke 4 buah kotak atau variabel berupa a, b, c , dan d , maka dibuatlah 4 buah bintang yang mewakili jumlah angka kita ($\star\star\star\star$). Kemudian, keempat kotak atau variabel tersebut akan dipisah menggunakan garis sebanyak $n - 1$ alias 3. Jika diilustrasikan, maka seperti berikut:

$$\begin{array}{c} _|_|_ \\ a|b|c|d \end{array}$$

Apabila dimasukkan tepat satu bintang dalam setiap kotak, maka jumlah bintang tiap kotak adalah 4, sehingga memenuhi syarat.

$$\begin{array}{c} \star|\star|\star|\star \\ a|b|c|d \end{array}$$

Apabila ditinjau, maka jumlah $a = b = c = d = 1$. Kemudian, salah satu cara lain dapat ditemukan dengan menggeser garis/menggeser bintang seperti berikut:

$$\begin{array}{c} \star\star| \quad |\star|\star \\ a|b|c|d \end{array}$$

Pada pola tersebut, jumlah bintang tetap 4, tetapi dengan $a = 2, b = 0, c = d = 1$. Sehingga tetap memenuhi syarat.

Secara umum, teorema bintang dan garis didefinisikan sebagai cara untuk menyusun n buah bilangan yang didistribusikan ke dalam k jumlah kotak. Kombinasi tiap kemungkinan bisa didapatkan dengan

melakukan pergeseran terhadap garis yang diwakilkan dengan $k - 1$ atau bintang yang diwakilkan dengan n . Maka, kita mencari banyak cara atau kombinasi untuk menyusun n buah bintang atau $k - 1$ buah garis dari n buah bintang yang ditambah dengan $k - 1$ buah garis atau sebagai berikut:

$$\frac{(n + k - 1)!}{n! (k - 1)!} = \binom{n + k - 1}{k - 1}$$

$$\binom{n + k - 1}{n} = \binom{n + k - 1}{k - 1}$$

Maka, pada kasus $n = 4$ dan $k = 4$, diperoleh banyak kombinasi adalah 35 cara. Pada kasus algoritma tersebut, diketahui bahwa n adalah bilangan masukan pengguna dan kotak pembagi pada persoalan tersebut (k) adalah 4 yang terdiri dari w, x, y , dan z . Maka, garis pemisah 4 kotak tersebut adalah $k - 1$ atau 3. Sehingga, banyak kombinasi dapat diketahui dengan menggunakan persamaan sebagai berikut:

$$\frac{(n + 3)!}{n! 3!} = \binom{n + 3}{3}$$

$$\binom{n + 3}{n} = \binom{n + 3}{3}$$

Dikarenakan operasi utama pada persamaan tersebut adalah faktorial, maka kemungkinan Big-O dari program tersebut adalah $O(n)$ dikarenakan program perlu untuk mengalikan dari 1 hingga n -kali. Namun, dikarenakan program ingin dibuat dengan kompleksitas waktu seminim mungkin atau $O(1)$, maka digunakan variabel lain sebagai batas perulangan pada program tersebut. Variabel yang digunakan adalah variabel $k - 1$ yang merepresentasikan banyak baris yang dihasilkan dari k buah kotak. Hal ini dikarenakan variabel k merupakan konstan dan telah ditentukan dari persoalan bernilai 4.

Dengan menggunakan definisi awal teorema bintang dan garis, maka dapat dibuat sebuah perulangan dengan menggunakan variabel $k - 1$ sebagai batas atas perulangan, tetapi diperlukan modifikasi pada variabel yang berfungsi sebagai penampung hasil perhitungan, yakni seperti berikut:

Karena $k - 1$ bernilai 3, maka perulangan hanya akan dijalankan 3 kali, yakni dari 1, 2, dan 3. Apabila mengacu pada penggunaan variabel N sebagai batas perulangan atau $N!$, maka akan menjadi seperti berikut:

$$N! = N. (N - 1). (N - 2)(N - 3) \dots 3.2.1$$

Apabila terdapat suatu bilangan r , maka $(N-r)!$ dapat ditulis sebagai

$$(N - r)! = (N - r). (N - r - 1)(N - r - 2) \dots 3.2.1$$

Karena $(N-r)!$ merupakan bagian dari $N!$, maka dapat dilakukan pencoretan terhadap beberapa anggota $N!$. Hal ini hanya berlaku apabila r merupakan sebuah bilangan bulat tak negatif yang kurang dari N , sehingga menjadi:

$$\frac{N!}{(N - r)!} = N. (N - 1). (N - 2) \dots (N - r + 1)$$

$$\frac{N!}{(N - r)!} = (N - r + 1)(N - r + 2) \dots N$$

Dengan jumlah perkalian sebanyak r kali. Kemudian, dengan melakukan substitusi $N = n + k - 1$ dan $r = k - 1$, didapat:

$$\frac{(n + k - 1)!}{(n + k - 1 - (k - 1))!} = (n + k - 1 - (k - 1) + 1)(n + k - 1 - (k - 1) + 2) \dots (n + k - 1)$$

$$\frac{(n + k - 1)!}{n!} = (n + 1). (n + 2) \dots (n + k - 1)$$

Kemudian, dilakukan perkalian kedua sisi persamaan dengan $\frac{1}{(k-1)!}$

$$\frac{(n+k-1)!}{n!(k-1)!} = \frac{(n+1).(n+2) \dots (n+k-1)}{(k-1)!}$$

Bentuk persamaan tersebut sangat mirip dengan persamaan pada teorema bintang dan garis, maka dapat dibuat menjadi:

$$\binom{n+k-1}{k-1} = \frac{(n+1).(n+2) \dots (n+k-1)}{(k-1)!}$$

Dengan mengembangkan $(k-1)!$ Dan melakukan substitusi $k = 4$, maka akan menjadi:

$$\begin{aligned} \binom{n+4-1}{4-1} &= \frac{(n+1).(n+2) \dots (n+4-1)}{(4-1)!} \\ \binom{n+3}{3} &= \frac{(n+1).(n+2)(n+3)}{3!} \\ \binom{n+3}{3} &= \frac{(n+1).(n+2)(n+3)}{3.2.1} \end{aligned}$$

Untuk penyesuaian dengan perulangan, maka dibuatlah sebuah variabel *increment* berupa i dengan nilai awal 1 yang akan berulang hingga $k-1$ kali atau 3 kali. Pada bagian pembilang, dapat disisipkan variabel *increment* i pada penjumlahan $n-r$. Hal ini dikarenakan setelah $n-r+1$ adalah $n-r+2$ dan $n-r+3$. Kemudian, pada bagian penyebut yang merupakan faktorial dari $k-1$ atau 3, dapat dibuat sebuah fungsi yang mengalikan variabel *increment* i selama perulangan terjadi.

Salah satu contoh pada penyelesaian permasalahan tersebut adalah dengan mengambil $n = 7$ dan $k = 4$, maka $r = k-1$ adalah 3. Maka, dengan menggunakan rumus teorema bintang dan garis akan menjadi:

$$\frac{(7+4-1)!}{7!(4-1)!} = \binom{7+4-1}{4-1}$$

$$\frac{10!}{7!3!} = \binom{10}{3}$$

$$\frac{10!}{7!3!} = \frac{10.9.8.7!}{7!.3.2.1}$$

Dengan melakukan pencoretan pada 7!, maka akan menjadi:

$$\frac{10!}{7!3!} = \frac{10.9.8}{3.2.1}$$

Maka yang tersisa pada pembilang adalah 8, 9, dan 10. Di mana 8 merupakan (7+1) dan 9 adalah (7+2), dan 10 adalah (7+3). Apabila ditulis dengan menggunakan variabel r , maka 7 adalah $n - r + 1$ alias $n + k - 1 - (k - 1) + 1$ atau $n + 1$. Pola ini sama persis dengan rumus yang telah diturunkan sebelumnya. Hal ini dikarenakan k sebagai jumlah kotak memiliki nilai yang sama, yakni 4. Maka, dapat disimpulkan bahwa persamaan penyelesaian permasalahan tersebut adalah:

$$\binom{n+3}{3} = \frac{(n+1).(n+2)(n+3)}{3.2.1}$$

Atau, dalam program, dapat dibuat variabel penampung pembilang sebagai:

$$atas = (N - r + i)$$

Dengan $N = n + k - 1$ dan $r = k - 1$, serta i merupakan variabel *increment* yang dimulai dari 1 hingga $k - 1$ alias 3. Pada bagian penyebut, dapat dibuat sebuah variabel penampung nilai penyebut sebagai berikut:

$$bawah = bawah * i$$

Dengan variabel bawah adalah variabel yang menampung hasil faktorial dari i yang dimulai dari 1 hingga $k - 1$ alias 3. Namun, dengan menggunakan variabel r atau $k - 1$ sebagai batas perulangan, terdapat sebuah permasalahan kecil, yakni batas perulangan yang tidak efisien. Anggaplah $n = 5$ dan $k = 4$, maka $r = 3$. Kombinasinya akan menjadi:

$$\binom{5}{3}$$

Sehingga, program akan berulang sebanyak 3 kali. Padahal, terdapat persamaan lain yang nilainya sama, yakni:

$$\binom{5}{3} = \binom{5}{2}$$

Apabila dikembangkan, maka akan menjadi:

$$\frac{5.4.3!}{(5-3)!3!} = \frac{5.4.3!}{2!(5-2)!}$$

$$\frac{5.4.3!}{2!3!} = \frac{5.4.3!}{2!3!}$$

Kedua kombinasi tersebut sama persis dengan perbedaan kombinasi pada sisi kiri memerlukan program untuk melakukan perulangan sebanyak 3 kali. Sedangkan, pada sisi kanan memerlukan program untuk melakukan perulangan sebanyak 2 kali. Maka sisi kanan lebih efisien. Untuk itu, dibuat sebuah fungsi struktur kontrol menggunakan *if* yang akan mengecek apabila pengurangan $n + k - 1$ dengan $k - 1$ lebih kecil dari $k - 1$ itu sendiri, maka pengurangan $n + k - 1$ dengan $k - 1$ akan menjadi batas perulangan.

Kemudian dibuat sebuah metode pengecekan sederhana untuk menolak masukan pengguna apabila nilai variabel n bernilai kurang dari 0 untuk menghindari kesalahan perhitungan. Selain itu, digunakan variabel *long* yang memiliki ukuran lebih besar dibanding *integer* untuk mencegah apabila hasil faktorial terlalu besar atau *overflow*.

Dengan menggunakan metode tersebut, maka kompleksitas waktu $T(n)$ dari program tersebut adalah:

$$T(n) = (k - 1)c$$

Dikarenakan k sudah diketahui bernilai 4, maka menjadi

$$T(n) = (4 - 1)c$$

$$T(n) = 3c$$

Maka, Big-O dari program tersebut adalah:

$$T(n) \approx O(3)$$

$$T(n) \approx O(1)$$

Program berhasil dimodifikasi sehingga memiliki Big-O sebesar $O(1)$, tetapi hal ini akan berlaku jika dan hanya jika nilai k sebagai banyak kotak pada persoalan tersebut sudah diketahui dan berupa konstan. Apabila nilai variabel k adalah masukan pengguna, maka Big-O dari program tersebut tetaplah $O(n)$. Hal ini dikarenakan program perlu melakukan perulangan hingga $n-1$ kali (dalam Big-O, konstan berupa -1 dihiraukan).

1.2 Deskripsi Program

1.2.1 Tugas 1

Proses berjalannya program:

1. Program dimulai dengan *start*.

2. Dikarenakan program tersebut adalah penggalan sebuah *method*, maka program langsung memasuki *method* “hitungKombinasi” yang memiliki pengembalian berupa *integer* dengan menggunakan variabel *n* sebagai parameter.
3. Program mendeklarasikan variabel kombinasi bertipe data *integer* yang diberi nilai awal 0.
4. Program memulai sebuah perulangan menggunakan *for* selagi mendeklarasikan variabel *w* sebagai variabel *increment* yang diberi nilai awal 0 dan akan terus bertambah dengan 1 selama nilainya kurang dari atau sama dengan *n*.
5. Selama perulangan berlangsung, program memulai sebuah perulangan bertingkat menggunakan *for* selagi mendeklarasikan variabel *x* sebagai variabel *increment* yang diberi nilai awal 0 dan akan terus bertambah dengan 1 selama nilainya kurang dari atau sama dengan *n*.
6. Selama perulangan berlangsung, program memulai sebuah perulangan bertingkat menggunakan *for* selagi mendeklarasikan variabel *y* sebagai variabel *increment* yang diberi nilai awal 0 dan akan terus bertambah dengan 1 selama nilainya kurang dari atau sama dengan *n*.
7. Selama perulangan berlangsung, program memulai sebuah perulangan bertingkat menggunakan *for* selagi mendeklarasikan variabel *z* sebagai variabel *increment* yang diberi nilai awal 0 dan akan terus bertambah dengan 1 selama nilainya kurang dari atau sama dengan *n*.
8. Kemudian, selama semua perulangan sebelumnya berlangsung, program memulai sebuah struktur kontrol menggunakan *if* yang akan mengecek apakah penjumlahan nilai variabel *w*, *x*, *y*, dan *z* hasilnya sama dengan *n*.
9. Apabila kondisi tersebut benar, maka nilai variabel kombinasi akan diperbarui dengan menambahkan nilai variabel kombinasi sebelumnya dengan 1. Apabila tidak, maka perulangan akan terus berjalan hingga selesai.
10. Semua perulangan akan berjalan selama kondisinya terpenuhi. Setelah semua perulangan berhenti, maka *method* atau program akan

memberikan pengembalian atau *return* berupa nilai variabel kombinasi.

11. Program selesai.

1.2.2 Tugas 2

Proses berjalannya program:

1. Program dimulai dengan *start*.
2. Sebuah perulangan dengan menggunakan *do-while* dimulai dengan *do* menyuruh pengguna untuk memasukkan sebuah bilangan bulat tak negatif yang akan disimpan di variabel *n*.
3. *While* akan mengecek masukan pengguna dan selama masukannya bernilai kurang dari 0, program akan terus menyuruh pengguna untuk memasukkan kembali nilai variabel *n*.
4. Program mendeklarasikan variabel *k* berupa *integer* dan memberikan nilai 4 ke variabel tersebut.
5. Program memanggil method “hitungKombinasiOptimal” yang tidak memiliki pengembalian atau bertipe *void* dengan menggunakan nilai variabel *n* yang merupakan masukan pengguna dan nilai variabel *k* sebagai parameter.
6. Program memasuki method “hitungKombinasiOptimal” dan mendeklarasikan variabel *N* bertipe data *integer* dan memberikan nilainya berupa $N = n + k - 1$, di mana *n* dan *k* adalah parameter pada *method* tersebut.
7. Program mendeklarasikan variabel *r* dengan tipe data *integer* dan memberikan nilainya berupa $r = k - 1$, dengan *k* merupakan parameter *method*.
8. Program memasuki struktur kontrol *if* dan mengecek apakah nilai variabel *r* lebih besar daripada nilai variabel $N - r$. Apabila kondisi tersebut benar, maka nilai variabel *r* diperbarui menjadi $r = N - r$. Apabila tidak, maka nilai variabel *r* tetap.
9. Program mendeklarasikan variabel *atas* dan *bawah* bertipe data *long* dan diberi nilai awal 1.

10. Program memulai sebuah perulangan menggunakan *for* sembari mendeklarasikan variabel *i* sebagai variabel yang akan melakukan *increment* selama perulangan berlangsung. Variabel *i* kemudian diberi nilai awal 1 dan terus bertambah dengan 1 selama nilainya kurang dari atau sama dengan *r*.
11. Selama perulangan berlangsung, program memperbarui nilai variabel *atas* dengan mengalikan nilai variabel *atas* sebelumnya dengan pengurangan nilai variabel *N* dengan *r* yang kemudian ditambah 1 atau

$$atas = atas * (N - r + i)$$

12. Kemudian, selama perulangan berlangsung juga, program akan memperbarui nilai variabel *bawah* dengan mengalikan nilai variabel *bawah* sebelumnya dengan nilai variabel *i* saat ini atau

$$bawah = bawah * i$$

13. Perulangan akan terus terjadi selama nilai variabel *i* kurang dari atau sama dengan *r*. Saat nilai variabel *i* lebih dari nilai variabel *r*, maka perulangan akan berhenti.
14. Setelah perulangan berhenti, program mendeklarasikan variabel *banyakCara* bertipe data *long* dengan nilai hasil pembagian *integer* antara nilai variabel *atas* dengan *bawah* atau

$$banyakCara = \frac{atas}{bawah}$$

15. Kemudian, dikarenakan *method* tidak memiliki pengembalian atau bersifat *void*, maka program mencetak variabel *banyakCara* dengan ("Total kombinasi penjumlahan " + *k* + " angka dari " + *n* + " adalah : " + *banyakCara*”, dengan *n* dan *k* adalah parameter pada *method* dan *banyakCara* adalah variabel hasil perhitungan yang telah dihitung.
16. Program selesai.

BAB II

SOURCE CODE

2.1 Tugas 1

```
public static int hitungKombinasi(int n) {
    int kombinasi = 0;
    for (int w = 0; w <= n; w++) {
        for (int x = 0; x <= n; x++) {
            for (int y = 0; y <= n; y++) {
                for (int z = 0; z <= n; z++) {
                    if (w + x + y + z == n) {
                        kombinasi++;
                    }
                }
            }
        }
    }
    return kombinasi;
}
```

2.2 Tugas 2

```
import java.util.Scanner;

public class modifikasiProgram {
    public static void main(String[] args) {

        Scanner masukan = new Scanner(System.in);

        System.out.println("Program penghitung kombinasi
penjumlahan 4 angka menggunakan teorema garis dan bintang");

        System.out.println();

        int n;

        do {
            System.out.print("Masukkan angka kunci kombinasi :
");
            n = masukan.nextInt();
        } while (n < 0);

        int k = 4;

        hitungKombinasiOptimal(n, k);

    }

    public static void hitungKombinasiOptimal(int n, int k) {

        int N = n + k - 1;
        int r = k - 1;

        if (r > N - r) {
            r = N - r;
        }
    }
}
```

```
    }

    long atas = 1;
    long bawah = 1;

    for (int i = 1; i <= r; i++) {
        atas *= (N - r + i);
        bawah *= i;
    }

    long banyakCara = atas / bawah;

    System.out.println("Total kombinasi penjumlahan " + k +
        " angka dari "+ n + " adalah : " + banyakCara);
    }
}
```

BAB III

OUTPUT PROGRAM

3.1 Output Tugas 1

3.1.1. Output apabila diberi masukan $n = 4$

```
run:
Masukkan n : 4
Total kombinasi 4 angka dari 4 adalah : 35
BUILD SUCCESSFUL (total time: 7 seconds)
```

3.1.2. Output apabila diberi masukan $n = 10$

```
run:
Masukkan n : 10
Total kombinasi 4 angka dari 10 adalah : 286
BUILD SUCCESSFUL (total time: 2 seconds)
```

3.1.3. Output apabila diberi masukan $n = 100$

```
run:
Masukkan n : 100
Total kombinasi 4 angka dari 100 adalah : 176851
BUILD SUCCESSFUL (total time: 2 seconds)
```

3.1.4. Output apabila diberi masukan $n = 0$

```
run:
Masukkan n : 0
Total kombinasi 4 angka dari 0 adalah : 1
BUILD SUCCESSFUL (total time: 2 seconds)
```

3.1.5. Output apabila diberi masukan $n = -1$

```
run:
Masukkan n : -1
Total kombinasi 4 angka dari -1 adalah : 0
BUILD SUCCESSFUL (total time: 2 seconds)
```

3.2 Output Tugas 2

3.2.1 Output apabila diberi masukan $n = 4$

```
run:
Program penghitung kombinasi penjumlahan 4 angka menggunakan teorema garis dan bintang

Masukkan angka kunci kombinasi : 4
Total kombinasi penjumlahan 4 angka dari 4 adalah : 35
BUILD SUCCESSFUL (total time: 1 minute 51 seconds)
```

3.2.2 Output apabila diberi masukan $n = 10$

```
run:
Program penghitung kombinasi penjumlahan 4 angka menggunakan teorema garis dan bintang

Masukkan angka kunci kombinasi : 10
Total kombinasi penjumlahan 4 angka dari 10 adalah : 286
BUILD SUCCESSFUL (total time: 2 seconds)
```

3.2.3 Output apabila diberi masukan $n = 100$

```
run:
Program penghitung kombinasi penjumlahan 4 angka menggunakan teorema garis dan bintang

Masukkan angka kunci kombinasi : 100
Total kombinasi penjumlahan 4 angka dari 100 adalah : 176851
BUILD SUCCESSFUL (total time: 2 seconds)
```

3.2.4 Output apabila diberi masukan $n = 0$

```
run:
Program penghitung kombinasi penjumlahan 4 angka menggunakan teorema garis dan bintang

Masukkan angka kunci kombinasi : 0
Total kombinasi penjumlahan 4 angka dari 0 adalah : 1
BUILD SUCCESSFUL (total time: 6 seconds)
```

3.2.5 Output apabila diberi masukan $n = -1$

```
run:
Program penghitung kombinasi penjumlahan 4 angka menggunakan teorema garis dan bintang

Masukkan angka kunci kombinasi : -1
Masukkan angka kunci kombinasi : -1
Masukkan angka kunci kombinasi : -1
Masukkan angka kunci kombinasi : -1
Masukkan angka kunci kombinasi : -1
```